

Machine Learning as a Mathematical Discovery Tool: an Application to the Langlands Program

Thomas Lanard
(joint work with Alberto Mínguez)
Seville, 13 May 2026

Seminar of Algebra, Universidad de Sevilla

Get the slides:



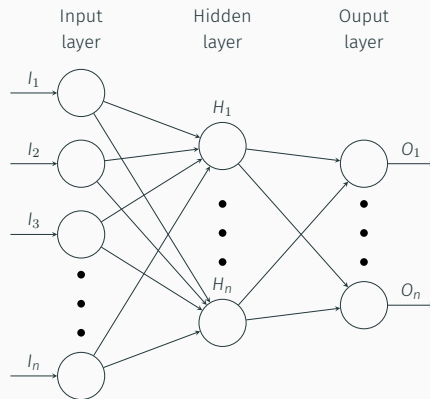
[https://www.thomaslanard.com/
slides/AZ.pdf](https://www.thomaslanard.com/slides/AZ.pdf)

Table of contents

- Machine learning
- Aubert–Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation

Table of contents

- Machine learning
- Aubert-Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation



- Machine learning
- Aubert–Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation

$$\begin{array}{ccc} \mathrm{Irr}(G) & \longrightarrow & \mathrm{Irr}(G) \\ \pi & \longmapsto & \hat{\pi} \end{array}$$

Table of contents

- Machine learning
- Aubert-Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation

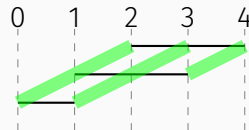


Table of contents

- Machine learning
- Aubert-Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation

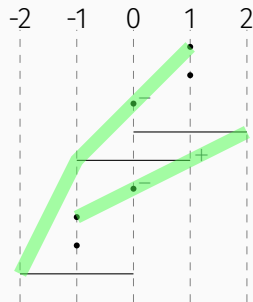
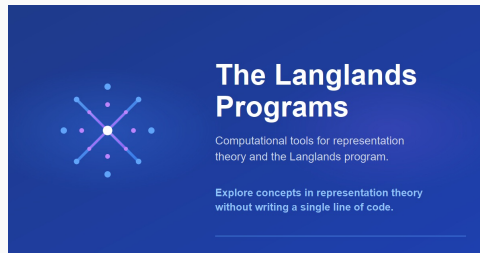


Table of contents

- Machine learning
- Aubert–Zelevinsky Involution
- The case of GL_n
- The case of classical groups
- Implementation

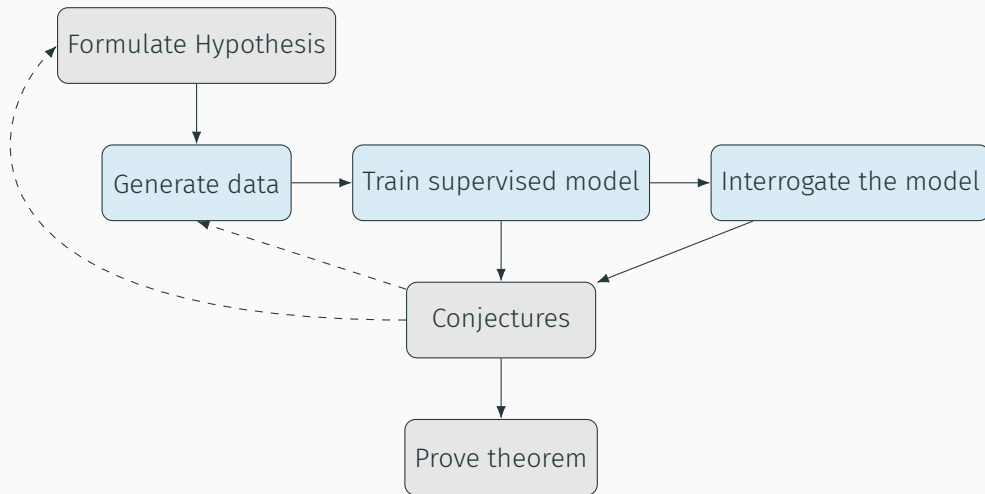


Machine learning

Motivation:

- AI excels at finding patterns in large datasets.
- In mathematics, generating large and precise datasets is straightforward.
- AI can be used to uncover new relations between mathematical objects.

Machine learning workflow



A **neural network** is a function $\varphi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ given by composition:

$$\varphi = \varphi_r \circ \cdots \circ \varphi_1$$

where each layer is:

$$\varphi_i : \mathbb{R}^{n_i} \xrightarrow{\text{linear}} \mathbb{R}^{n_{i+1}} \xrightarrow{\text{ReLU}} \mathbb{R}^{n_{i+1}}$$

ReLU (Rectified Linear Unit): $x \mapsto \max(0, x)$ applied componentwise

(no ReLU on the last layer φ_r)

Supervised learning

Goal: Approximate an unknown function $\hat{\varphi} : \mathbb{R}^n \rightarrow \mathbb{R}^m$

Training data: Samples $(x_i, \hat{\varphi}(x_i))_{i=1,\dots,N}$

Loss function: Measures prediction error

$$l(\varphi) = \frac{1}{N} \sum_{i=1}^N \|\varphi(x_i) - \hat{\varphi}(x_i)\|^2$$

Training: Use **gradient descent** to find parameters that minimize $l(\varphi)$

Example: Knot theory (Davies et al., 2021)

Advancing mathematics by guiding human intuition with AI, Davies et al., Nature 2021

Setup: For a knot K , two types of invariants:

- **Geometric** $X(K)$: volume, cusp shape, injectivity radius, ...
- **Algebraic** $Y(K)$: signature $\sigma(K)$, Jones polynomial, ...

Question: Is there a hidden relation between $X(K)$ and $Y(K)$?

Step 1 – Existence of a pattern:

- Train a neural network to predict **all algebraic invariants** from $X(K)$
- Good accuracy for $\sigma(K)$

Example: Knot theory (Davies et al., 2021)

Step 2 – Understanding the pattern:

- Apply **gradient saliency** to identify the most relevant features
- Most salient:
 - $\text{vol}(K)$ the **volume**
 - μ the **meridional translation**
 - λ the **longitudinal translation**

Example: Knot theory (Davies et al., 2021)

Step 2 – Understanding the pattern:

- Apply **gradient saliency** to identify the most relevant features
- Most salient:
 - $\text{vol}(K)$ the **volume**
 - μ the **meridional translation**
 - λ the **longitudinal translation**

Step 3 – Theorem:

Let $\text{slope}(K) = \text{Re}(\lambda/\mu)$. There exists $c > 0$ such that for any hyperbolic knot K ,

$$|2\sigma(K) - \text{slope}(K)| \leq c \cdot \text{vol}(K) \cdot \text{inj}(K)^{-3}$$

Aubert–Zelevinsky involution

An involution for finite reductive groups

Unipotent characters of $\mathrm{GL}_n(\mathbb{F}_q)$ are parametrized by partitions of n .

There is a natural involution given by transpose of partitions.

Theorem (Alvis–Curtis duality)

This involution is the Alvis–Curtis duality for $\mathrm{GL}_n(\mathbb{F}_q)$.

An involution for finite reductive groups

Unipotent characters of $\mathrm{GL}_n(\mathbb{F}_q)$ are parametrized by partitions of n .

There is a natural involution given by transpose of partitions.

Theorem (Alvis–Curtis duality)

This involution is the Alvis–Curtis duality for $\mathrm{GL}_n(\mathbb{F}_q)$.

$\mathrm{GL}_n(\mathbb{Q}_p)$		$\mathrm{GL}_n(\mathbb{F}_q)$
Aubert–Zelevinsky	$\longleftrightarrow$	Alvis–Curtis
Multisegments	$\longleftrightarrow$	Partitions
Mœglin–Waldspurger	$\longleftrightarrow$	Transpose

Setting:

- F a p -adic field with norm $|\cdot|$
- G a p -adic group

e.g. $F = \mathbb{Q}_p$, $|p^k u|_p = p^{-k}$

e.g. $G = \mathrm{GL}_n(\mathbb{Q}_p)$ or $\mathrm{Sp}_{2n}(\mathbb{Q}_p)$

Representations:

- $\mathrm{Rep}(G)$ = smooth complex finite length representations
- $\mathrm{Irr}(G)$ = irreducible representations

The Aubert–Zelevinsky involution

The Aubert–Zelevinsky involution is a map:

$$\begin{array}{ccc} \mathrm{Irr}(G) & \longrightarrow & \mathrm{Irr}(G) \\ \pi & \longmapsto & \widehat{\pi} \end{array}$$

The Aubert–Zelevinsky involution

The Aubert–Zelevinsky involution is a map:

$$\begin{array}{ccc} \mathrm{Irr}(G) & \longrightarrow & \mathrm{Irr}(G) \\ \pi & \longmapsto & \widehat{\pi} \end{array}$$

Key properties:

- The map $\pi \mapsto \widehat{\pi}$ is an involution on $\mathrm{Irr}(G)$.

The Aubert–Zelevinsky involution

The Aubert–Zelevinsky involution is a map:

$$\begin{aligned}\mathrm{Irr}(G) &\longrightarrow \mathrm{Irr}(G) \\ \pi &\longmapsto \widehat{\pi}\end{aligned}$$

Key properties:

- The map $\pi \mapsto \widehat{\pi}$ is an involution on $\mathrm{Irr}(G)$.
- It exchanges the trivial and the Steinberg.

$$\widehat{\mathbf{1}}_G = \mathrm{St}_G \quad \text{and} \quad \widehat{\mathrm{St}}_G = \mathbf{1}_G$$

The case of GL_n

Definition

Let $n_1, n_2 \in \mathbb{N}^*$. The **parabolic induction** functor is

$$\begin{array}{ccc} \times: & \text{Rep}(GL_{n_1}) \times \text{Rep}(GL_{n_2}) & \longrightarrow \text{Rep}(GL_{n_1+n_2}) \\ & (\pi_1, \pi_2) & \longmapsto \pi_1 \times \pi_2 \end{array}$$

Definition

A irreducible representation π is **cuspidal** if it does not appear as a sub-quotient of any parabolic induction $\pi_1 \times \pi_2$.

Notation: $\text{Cusp}(GL)$ denotes the set of isomorphism classes of cuspidal representations.

Definition

A **segment** is a subset of $\text{Cusp}(\text{GL})$ of the form

$$\Delta = \{\rho| \cdot |^x, \rho| \cdot |^{x+1}, \dots, \rho| \cdot |^y\}$$

where $\rho \in \text{Cusp}(\text{GL})$, and $x \leq y$ with $y - x \in \mathbb{N}$.

We use the notation $\Delta = [x, y]_\rho$.

Example: $\Delta = [0, 2]_\rho = \{\rho, \rho| \cdot |, \rho| \cdot |^2\}$

Operations on segments

For a segment $\Delta = [x, y]_\rho$ (with ρ unitary):

- $e(\Delta) = y$ (the **end** of Δ)
- $b(\Delta) = x$ (the **beginning** of Δ)
- $c(\Delta) = \frac{x+y}{2}$ (the **center** of Δ)

Let $\Delta^- = [x, y-1]_\rho$ (removing the last element)

Example: For $\Delta = [0, 2]_\rho$:

- $e(\Delta) = 2, b(\Delta) = 0, c(\Delta) = 1$
- $\Delta^- = [0, 1]_\rho = \{\rho, \rho | \cdot | \}$

Definition

A **multisegment** is a finite formal sum of segments:

$$\mathfrak{m} = \Delta_1 + \cdots + \Delta_r.$$

We denote by Mult the set of all multisegments.

Example: $\mathfrak{m} = [2, 3]_{\rho_1} + [2, 3]_{\rho_1} + [0, 2]_{\rho_1} + [2, 2]_{\rho_2}$ is a multisegment.

Theorem (Zelevinsky 1980)

To every segment $\Delta = [x, y]_\rho$, we can associate two irreducible representations $Z(\Delta)$ and $L(\Delta)$:

- $Z(\Delta)$ is the unique irreducible **subrepresentation** of the induced representation

$$\rho| \cdot |^x \times \cdots \times \rho| \cdot |^y$$

- $L(\Delta)$ is the unique irreducible **quotient** of this induced representation.

Theorem (Zelevinsky 1980)

To every multisegment $\mathbf{m} = \Delta_1 + \cdots + \Delta_r$, such that $c(\Delta_1) \geq \cdots \geq c(\Delta_r)$, we associate:

- $Z(\mathbf{m})$: unique irreducible subrepresentation of $Z(\Delta_1) \times \cdots \times Z(\Delta_r)$
- $L(\mathbf{m})$: unique irreducible quotient of $L(\Delta_1) \times \cdots \times L(\Delta_r)$

$Z(\mathbf{m})$ and $L(\mathbf{m})$ depend only on $\mathbf{m}$.

The maps $\mathbf{m} \mapsto Z(\mathbf{m})$ and $\mathbf{m} \mapsto L(\mathbf{m})$ give **bijections**

$$\text{Mult} \xrightarrow{\sim} \text{Irr}(\text{GL}).$$

Examples :

Multisegment $\mathbf{m}$	$Z(\mathbf{m})$	$L(\mathbf{m})$
$\left[-\frac{1}{2}, \frac{1}{2}\right]$	$\mathbf{1}_{\text{GL}_2}$	St_{GL_2}
$\left[-\frac{1}{2}, -\frac{1}{2}\right] + \left[\frac{1}{2}, \frac{1}{2}\right]$	St_{GL_2}	$\mathbf{1}_{\text{GL}_2}$

Examples :

Multisegment $\mathbf{m}$	$Z(\mathbf{m})$	$L(\mathbf{m})$
$\left[-\frac{1}{2}, \frac{1}{2}\right]$	$\mathbf{1}_{\text{GL}_2}$	St_{GL_2}
$\left[-\frac{1}{2}, -\frac{1}{2}\right] + \left[\frac{1}{2}, \frac{1}{2}\right]$	St_{GL_2}	$\mathbf{1}_{\text{GL}_2}$

Relation between $Z(\mathbf{m})$ and $L(\mathbf{m})$

$$\widehat{Z(\mathbf{m})} = L(\mathbf{m})$$

Question

Given a multisegment $\mathfrak{m}$, find $\mathfrak{m}^t$ such that

$$\widehat{Z(\mathfrak{m})} = Z(\mathfrak{m}^t).$$

An explicit algorithm was given by [Mœglin and Waldspurger](#).

Reduction to cuspidal lines

The algorithm works on each **cuspidal line** independently.

Definition

A **cuspidal line** is

$$\mathbb{Z}_\rho = \{\rho | \cdot |^n, n \in \mathbb{Z}\}$$

for a fixed cuspidal representation $\rho \in \text{Cusp}(\text{GL})$.

Let Mult_ρ = multisegments supported on $\mathbb{Z}_\rho$.

Then $\text{Mult} = \bigoplus_\rho \text{Mult}_\rho$ (where the sum runs over cuspidal lines), and this decomposition is compatible with duality:

$$\mathfrak{m} = \sum_\rho \mathfrak{m}_\rho \implies \mathfrak{m}^t = \sum_\rho \mathfrak{m}_\rho^t$$

Definition (Order on segments)

For segments on the same cuspidal line:

$$[x, y]_{\rho} \leq [x', y']_{\rho} \quad \text{if} \quad \begin{cases} x < x' \\ \text{or } x = x' \text{ and } y \geq y' \end{cases}$$

 This is NOT the lexicographic order!

Example: $[0, 2]_{\rho} \leq [1, 6]_{\rho} \leq [1, 3]_{\rho}$

The Møglin–Waldspurger algorithm

Let $\mathfrak{m} \in \text{Mult}_\rho$ be a multisegment.

Step 1: Find $e_{\max} = \max\{e(\Delta) \mid \Delta \in \mathfrak{m}\}$.

Step 2: Let Δ_1 be the largest segment (for $\leq$) with $e(\Delta_1) = e_{\max}$.

Step 3: Define inductively $\Delta_1 \geq \Delta_2 \geq \dots \geq \Delta_l$ where Δ_{j+1} is the largest segment such that:

- $\Delta_{j+1} \leq \Delta_j$
- $e(\Delta_{j+1}) = e(\Delta_j) - 1$

(we stop if no such segment exist)

The Mœglin–Waldspurger algorithm

Having found $\Delta_1, \dots, \Delta_l$, define:

- The **first segment of the dual**:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

- A **new multisegment**:

$$\mathfrak{m}^\# = \mathfrak{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

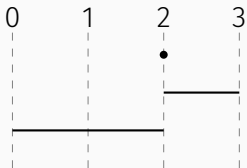
Theorem (Mœglin–Waldspurger 1986)

The dual multisegment is given recursively by:

$$\mathfrak{m}^t = \Delta'_1 + (\mathfrak{m}^\#)^t$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$\mathbf{m}^t = \dots$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

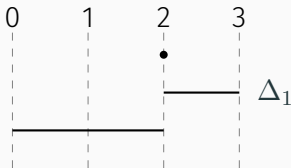
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

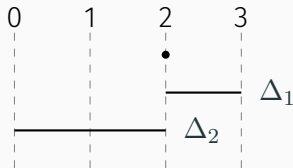
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = \dots$$

Reminder: MW algorithm

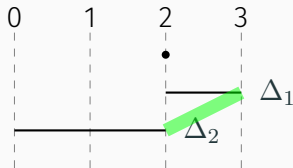
1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$
$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

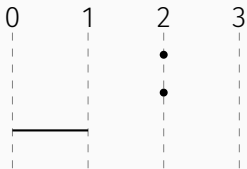
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

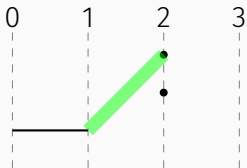
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

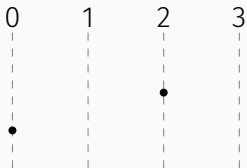
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

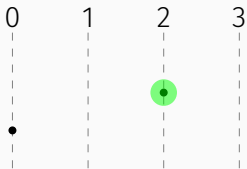
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + \dots$$

Reminder: MW algorithm

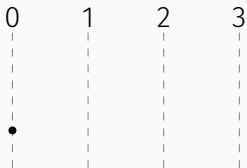
1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$
$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + \dots$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

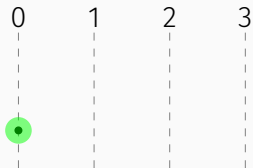
Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + [0, 0]$$

Reminder: MW algorithm

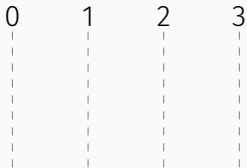
1. $e_{\max} = \max\{e(\Delta)\}$
2. $\Delta_1 =$ largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$
$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 1

Compute the dual of $\mathbf{m} = [2, 3] + [0, 2] + [2, 2]$.



Dual multisegment:

$$\mathbf{m}^t = [2, 3] + [1, 2] + [2, 2] + [0, 0]$$

Reminder: MW algorithm

1. $e_{\max} = \max\{e(\Delta)\}$
2. Δ_1 = largest segment with $e(\Delta_1) = e_{\max}$
3. $\Delta_1 \geq \dots \geq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$

Result:

$$\Delta'_1 = [e(\Delta_l), e(\Delta_1)]_\rho$$

$$\mathbf{m}^\# = \mathbf{m} - \sum_{j=1}^l (\Delta_j - \Delta_j^-)$$

Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



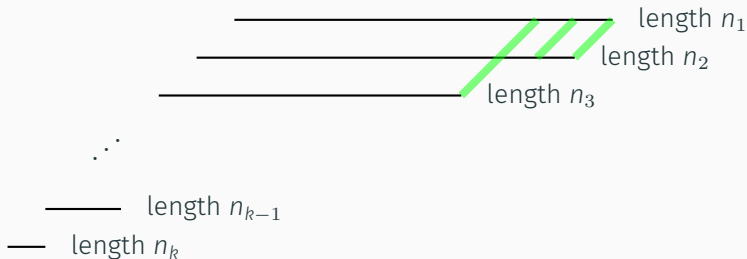
Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



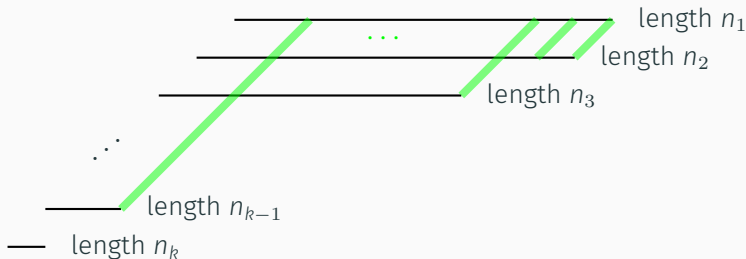
Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



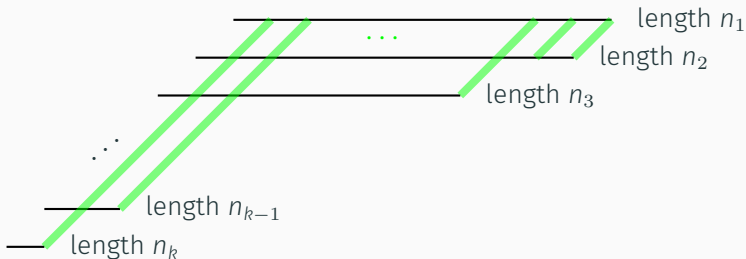
Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



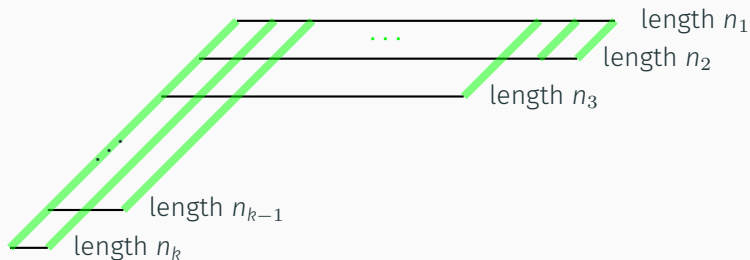
Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



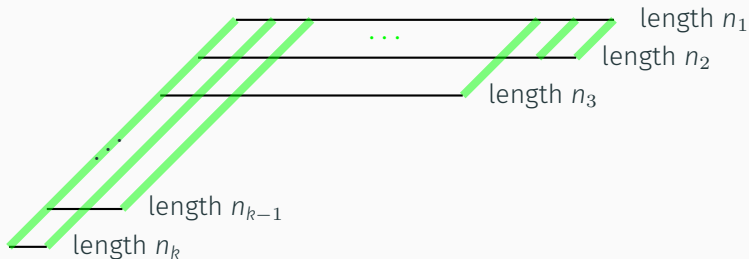
Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



Example 2: Transpose of a partition

Let $n_1 \geq \dots \geq n_k$ be a partition. Consider the multisegment $\mathbf{m}$ with segments of lengths $n_1, \dots, n_k$ and starting points decreasing by 1:



Then $\mathbf{m}^t$ corresponds to the **transpose partition** (n_i^*) .

The case of classical groups

Classification of irreducible representations

We now consider $G = \mathrm{Sp}_{2n}(F)$ or $\mathrm{SO}_{2n+1}(F)$.

Classification via local Langlands correspondence

Irreducible representations of G are parametrized by **Langlands data**

$$\pi = L(\mathfrak{m}, \phi, \eta)$$

Components of the Langlands data:

- **Multisegment:** $\mathfrak{m} = \Delta_1 + \cdots + \Delta_r$, with $c(\Delta_i) < 0$
- **Langlands parameter:** $\phi = \bigoplus_{i=1}^k \rho_i \boxtimes S_{a_i}$, where $\rho_i \in \mathrm{Cusp}(\mathrm{GL})$ and $a_i \in \mathbb{N}^*$
- **Character:** η assigns signs ± 1 to each $\rho_i \boxtimes S_{a_i}$

(satisfying technical conditions)

Question

Given $\pi = L(\mathfrak{m}, \phi, \eta) \in \text{Irr}(G)$, find $(\mathfrak{m}', \phi', \eta') \in \text{Data}(G)$ such that

$$\widehat{\pi} = L(\mathfrak{m}', \phi', \eta').$$

Atobe and Mínguez (2023) developed an algorithm for computing the Aubert–Zelevinsky dual using ρ -derivatives.

Machine learning:

- We use supervised learning, relying on the Atobe–Mínguez algorithm, to gain intuition about the Aubert–Zelevinsky duality.
- We began this process without making any mathematical assumptions or conjectures, in order to explore what structures the machine learning model might reveal on its own.

First attempt: Predict the entire dual

Question

Can we predict the entire dual $AD(\mathbf{m}, \phi, \eta)$ directly?

First attempt: Predict the entire dual

Question

Can we predict the entire dual $AD(\mathbf{m}, \phi, \eta)$ directly?

Result

The model shows **poor accuracy**.

First attempt: Predict the entire dual

Question

Can we predict the entire dual $AD(\mathbf{m}, \phi, \eta)$ directly?

Result

The model shows **poor accuracy**.

New strategy: Instead of predicting the entire dual at once, try to predict **one segment**.

Refined approach: Predict one segment

Question

Which segment should we predict first?

→ Live demo

Symmetrization of the data

- For $\Delta = [x, y]_\rho$, define $\Delta^\vee = [-y, -x]_{\rho^\vee}$
- By linearity, it extends to Mult

Example: For $\mathfrak{m} = [-1, 2] + [-1, 1]$ we have $\mathfrak{m}^\vee = [-2, 1] + [-1, 1]$

Let $\text{Symm} = \{\mathfrak{s} \in \text{Mult} \mid \mathfrak{s}^\vee = \mathfrak{s}\}$ (symmetric multisegments).

Define Symm^ε (symmetric multisegments with signs) as pairs $(\mathfrak{s}, \varepsilon)$ where:

- $\mathfrak{s} \in \text{Symm}$
- ε assigns signs ± 1 to centered segments (those with $c(\Delta) = 0$)

Example : $\mathfrak{s} = [-1, 2] + [-2, 1] + [-1, 1]$ avec $\varepsilon([-1, 1]) = -1$

From Langlands data to symmetric multisegment:

$$\begin{aligned}\mathrm{Data}(G) &\longrightarrow \mathrm{Symm}^\varepsilon \\ (\mathfrak{m}, \phi, \eta) &\longmapsto (\mathfrak{s}, \varepsilon)\end{aligned}$$

where the symmetric multisegment is

$$\mathfrak{s} = \sum_{\Delta \in \mathfrak{m}} (\Delta + \Delta^\vee) + \sum_{\rho \boxtimes S_a \in \phi} \left[-\frac{a-1}{2}, \frac{a-1}{2} \right]_\rho$$

and the signs are given by $\varepsilon \left(\left[-\frac{a-1}{2}, \frac{a-1}{2} \right]_\rho \right) = \eta(\rho \boxtimes S_a)$

Notation : $\pi = L(\mathfrak{s}, \varepsilon)$

Example of the transfer map

Langlands data:

$$(\mathfrak{m}, \phi, \eta) = ([-2, 1], S_1 + S_1 + S_3, (-1, -1, +1))$$

Example of the transfer map

Langlands data:

$$(\mathfrak{m}, \phi, \eta) = ([-2, 1], S_1 + S_1 + S_3, (-1, -1, +1))$$

Symmetrisation:


$$\mathfrak{s} = [-2, 1] + [-1, 2] + [0, 0] + [0, 0] + [-1, 1]$$

with $\varepsilon([0, 0]) = -1$ and $\varepsilon([-1, 1]) = +1$.

The AD algorithm

We define an involution

$$\text{AD} : \text{Symm}^\varepsilon \longrightarrow \text{Symm}^\varepsilon$$

Theorem (L.-Mínguez)

Let $\pi = L(\mathfrak{s}, \varepsilon) \in \text{Irr}(G)$. Then

$$\widehat{\pi} = L(\text{AD}(\mathfrak{s}, \varepsilon)).$$

The AD algorithm

We define an involution

$$\text{AD} : \text{Symm}^\varepsilon \longrightarrow \text{Symm}^\varepsilon$$

Theorem (L.-Mínguez)

Let $\pi = L(\mathfrak{s}, \varepsilon) \in \text{Irr}(G)$. Then

$$\widehat{\pi} = L(\text{AD}(\mathfrak{s}, \varepsilon)).$$

Remark: It is not easy to find an involution on Symm^ε !

Three types of cuspidal representations

- AD is defined on each cuspidal line $\mathbb{Z}_\rho$
- Unlike GL_n , the cuspidal ρ matters!
- Three cases depending on ρ : ugly, bad, good

Structure of the AD algorithm

AD is defined **recursively**:

For $(\mathfrak{s}, \varepsilon) \in \text{Symm}^\varepsilon$, construct:

- $(\mathfrak{s}_1, \varepsilon_1)$: first piece of the dual
- $(\mathfrak{s}^\#, \varepsilon^\#)$: new symmetric multisegment

such that

$$\text{AD}(\mathfrak{s}, \varepsilon) = (\mathfrak{s}_1, \varepsilon_1) + \text{AD}(\mathfrak{s}^\#, \varepsilon^\#)$$

AD algorithm (good case)

Let $(\mathfrak{s}, \varepsilon) \in \text{Symm}^\varepsilon$.

Step 1: Order segments by a specific order $\preceq$.

Step 2: Construct descending chain $\Delta_1 \succeq \cdots \succeq \Delta_r$ where:

- Δ_1 is the largest segment with maximal end
- Δ_{j+1} is the largest segment such that:
 1. $\Delta_{j+1} \preceq \Delta_j$
 2. $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 3. **Sign condition:** if $c(\Delta_{j+1}) = c(\Delta_j) = 0$ then $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Step 3: Extract $\mathfrak{s}_1$ from ends of Δ_j and beginnings of $\Delta_j^\vee$.

Step 4: Compute $\mathfrak{s}^\#$ by removing these parts + adjust signs.

Example

Setting: $G = \mathrm{Sp}_{2n}(F)$ and $\rho = 1$ (good case)

Langlands datum:

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Example

Setting: $G = \mathrm{Sp}_{2n}(F)$ and $\rho = 1$ (good case)

Langlands datum:

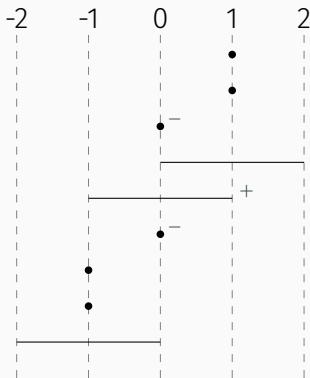
$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symmetrization:

$$\mathfrak{s} = [-2, 0] + [0, 2] + [-1, -1] + [1, 1] + [-1, -1] + [1, 1] + [0, 0] + [0, 0] + [-1, 1]$$

with $\varepsilon([0, 0]) = -1$ and $\varepsilon([-1, 1]) = 1$.

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

...

Reminder: AD algorithm

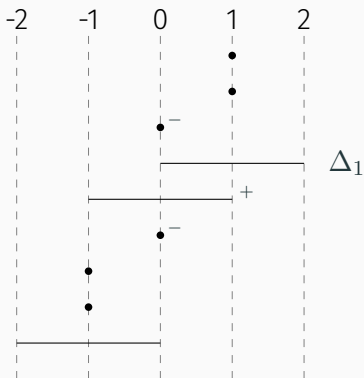
1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

...

Reminder: AD algorithm

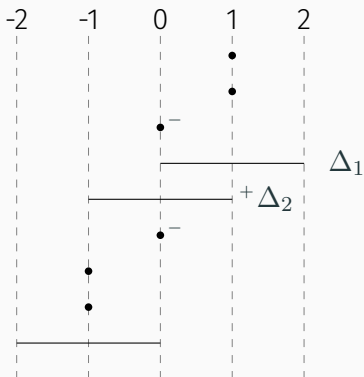
1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

...

Reminder: AD algorithm

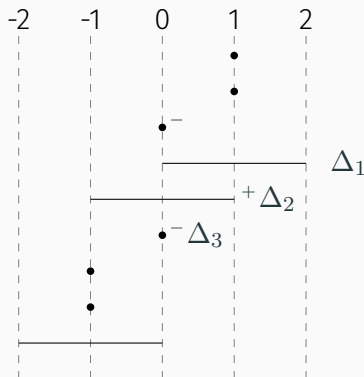
1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

...

Reminder: AD algorithm

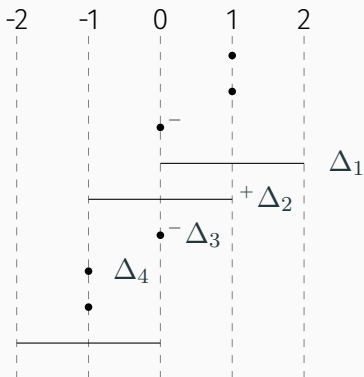
1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

...

Reminder: AD algorithm

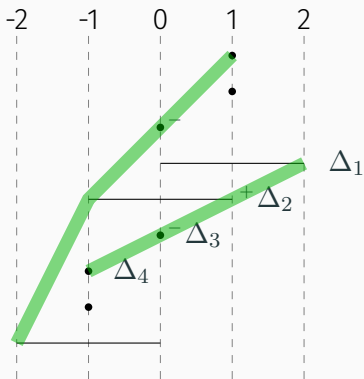
1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + \dots$

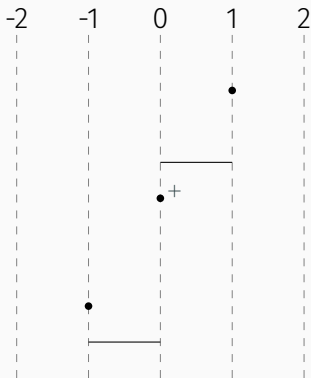
Reminder: AD algorithm

1. Sort by $\preceq$
2. $\Delta_1 =$ largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1 =$ formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$
 $\mathfrak{m}^\# =$ removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + \dots$

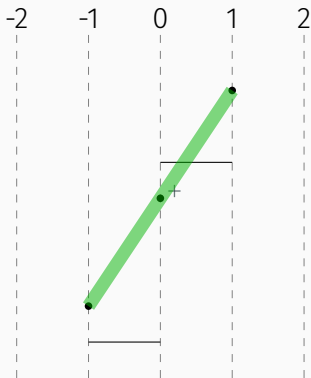
Reminder: AD algorithm

1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succeq \dots \succeq \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + [-1, 1] + \dots$
with $\varepsilon([-1, 1]) = 1$

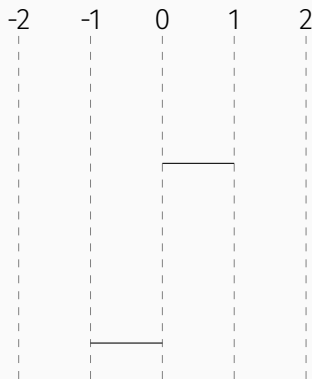
Reminder: AD algorithm

1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$
 $[-1, 2] + [-2, 1] + [-1, 1] + \dots$
with $\varepsilon([-1, 1]) = 1$

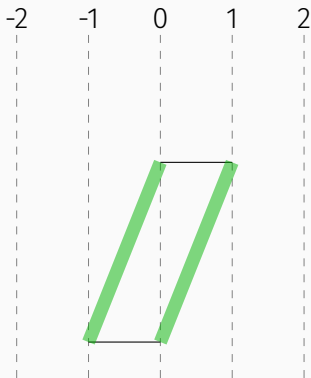
Reminder: AD algorithm

1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$
 $\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

$$[-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$$

with $\varepsilon([-1, 1]) = 1$

Reminder: AD algorithm

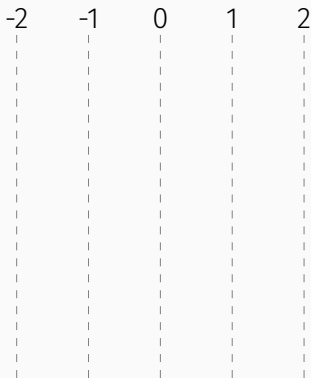
1. Sort by $\preceq$
2. $\Delta_1 =$ largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1 =$ formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\# =$ removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Step by step



Dual: $AD(\mathfrak{s}, \varepsilon) =$

$$[-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0]$$

with $\varepsilon([-1, 1]) = 1$

Reminder: AD algorithm

1. Sort by $\preceq$
2. Δ_1 = largest segment with $e(\Delta_1)$ maximal
3. $\Delta_1 \succ \dots \succ \Delta_l$ inductively:
 Δ_{j+1} largest segment with
 - $\Delta_{j+1} \leq \Delta_j$
 - $e(\Delta_{j+1}) = e(\Delta_j) - 1$
 - $\varepsilon(\Delta_{j+1})\varepsilon(\Delta_j) = -1$

Result:

$\mathfrak{s}_1$ = formed by $e(\Delta_i)$ and $b(\Delta_j^\vee)$

$\mathfrak{m}^\#$ = removal of $e(\Delta_i)$ and $b(\Delta_j^\vee)$

Example: Result

Input:

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Example: Result

Input:

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symmetrize + apply AD:

$$\text{AD}(\mathfrak{s}, \varepsilon) = ([-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0], \varepsilon')$$

with $\varepsilon'([-1, 1]) = 1$.

Example: Result

Input:

$$\pi = L([-2, 0] + [-1, -1] + [-1, -1], S_1 + S_1 + S_3, (-1, -1, 1))$$

Symmetrize + apply AD:

$$\text{AD}(\mathfrak{s}, \varepsilon) = ([-1, 2] + [-2, 1] + [-1, 1] + [0, 1] + [-1, 0], \varepsilon')$$

with $\varepsilon'([-1, 1]) = 1$.

Converting back to Langlands data:

$$\hat{\pi} = L([-2, 1] + [-1, 0], S_3, (1))$$

Comparison with Atobe–Mínguez algorithm

Atobe–Mínguez algorithm relies on ρ -derivatives.

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\mathbf{m} = [-2, 0] + 2[-1, -1]$$

$$\phi = S_1 + S_1 + S_3$$

$$\eta = (-1, -1, 1)$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ll} \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|^{-1}} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 & \longrightarrow \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & \eta = (-1, -1, 1) \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ccccc} \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|-1} & \mathbf{m} = [0, -2] & D_{L([-1, 0])} & \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ccccc} \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|-1} & \mathbf{m} = [0, -2] & D_{L([-1, 0])} & \mathbf{m} = [-2, -2] & D_{|\cdot|-2} & \mathbf{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ccccccc}
 \mathbf{m} = [-2, 0] + 2[-1, -1] & D_{|\cdot|^{-1}} & \mathbf{m} = [0, -2] & D_{L([-1, 0])} & \mathbf{m} = [-2, -2] & D_{|\cdot|^{-2}} & \mathbf{m} = \emptyset \\
 \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 & \longrightarrow & \phi = S_1 + S_1 + S_3 \\
 \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) & & \eta = (-1, -1, 1) \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & & & & & \mathbf{m} = [0, -1] \\
 & & & & & & \phi = S_1 \\
 & & & & & & \eta = (1)
 \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ccccc}
 \begin{array}{l} \mathbf{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathbf{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & & & & & \begin{array}{l} \mathbf{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array} \\
 & & & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & &
 \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

$$\begin{array}{ccccccc}
 \begin{array}{l} \mathbf{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathbf{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 & & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xleftarrow{S_{Z([0, 1])}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathbf{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array}
 \end{array}$$

Comparison with Atobe-Mínguez algorithm

Atobe-Mínguez algorithm relies on ρ -derivatives.

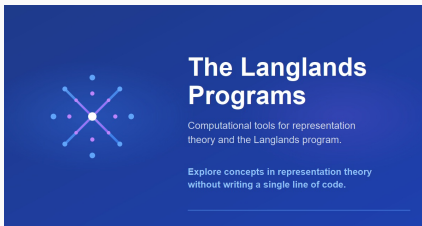
$$\begin{array}{ccccccc}
 \begin{array}{l} \mathbf{m} = [-2, 0] + 2[-1, -1] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-1}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{L([-1, 0])}} & \begin{array}{l} \mathbf{m} = [-2, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xrightarrow{D_{|\cdot|-2}} & \begin{array}{l} \mathbf{m} = \emptyset \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} \\
 & & & & & & \downarrow \pi \mapsto \hat{\pi} \\
 \begin{array}{l} \mathbf{m} = [0, -1] + [1, -2] \\ \phi = S_3 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|}^{(2)}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 + S_1 + S_3 \\ \eta = (-1, -1, 1) \end{array} & \xleftarrow{S_{Z([0, 1])}} & \begin{array}{l} \mathbf{m} = [0, -2] \\ \phi = S_1 \\ \eta = (1) \end{array} & \xleftarrow{S_{|\cdot|^2}} & \begin{array}{l} \mathbf{m} = [0, -1] \\ \phi = S_1 \\ \eta = (1) \end{array}
 \end{array}$$

Implementation

Implementation

Implementations:

- **Python/Sage package:**
`github.com/ThomasLanard/aubert-zelevinsky-duality`
- **Web application** (with Petar Bakić and Elad Zelingher):
`langlandsprograms.com`



Thank you for your attention!